

Document Update & Versioning

Addendum to the Autonomous Knowledge Engineering System Brief

Extends: AKES Architecture & Design Brief, June 2026

Architecture Addendum — June 2026

Contents

- A1.** Overview & Design Intent
- A2.** Document Version Lifecycle — State Machine
- A3.** How Updates Are Triggered
- A4.** The Three Update Paths (Agent / Human / Conflict)
- A5.** Version Store — Schema & Provenance
- A6.** Changelog Rollup & Consumer-Facing Views
- A7.** Partial Updates — Section-Level Granularity
- A8.** Version Retention & Compaction Policy
- A9.** Design Decisions (ADR-style)
- A10.** Integration with the Governance & Trust Model

A1. Overview & Design Intent

The main AKES brief describes what knowledge artifacts exist, how agents produce them, and how they are trusted. This addendum answers a more operational question: **once a document artifact exists, how is it updated when the world changes — and how are successive versions managed so that history is never lost and consumers always know what version they are reading?**

Three properties the versioning design must satisfy:

1. **Immutability of published versions.** Once a version reaches `Current` status, its content must not be silently changed. Changes produce a new version. This means any reader — human or agent — can always retrieve exactly what was published at a given date and trace why it changed.
2. **Diff-based, not full-rewrite updates.** A change to one service's dependency should not regenerate an entire Architecture Knowledge Pack. The Documentation Agent targets only the affected section(s), preserving human edits in unaffected sections.
3. **Bidirectionality.** The system reads from existing tools (GitHub, Confluence, Slack) and writes back to them. When a human edits a document in the published surface, that edit must flow back into the version store and the knowledge graph — not be silently overwritten the next time an agent processes a related change.

Key design rule: Human edits from the owning team always win over a concurrent agent-proposed draft. The agent's draft is discarded and re-evaluated against the human-edited state — it never silently overwrites human intent.

A2. Document Version Lifecycle — State Machine

Every version of every artifact moves through a defined set of states. Transitions are triggered by agent decisions, human actions, or drift detection events — never by time alone.

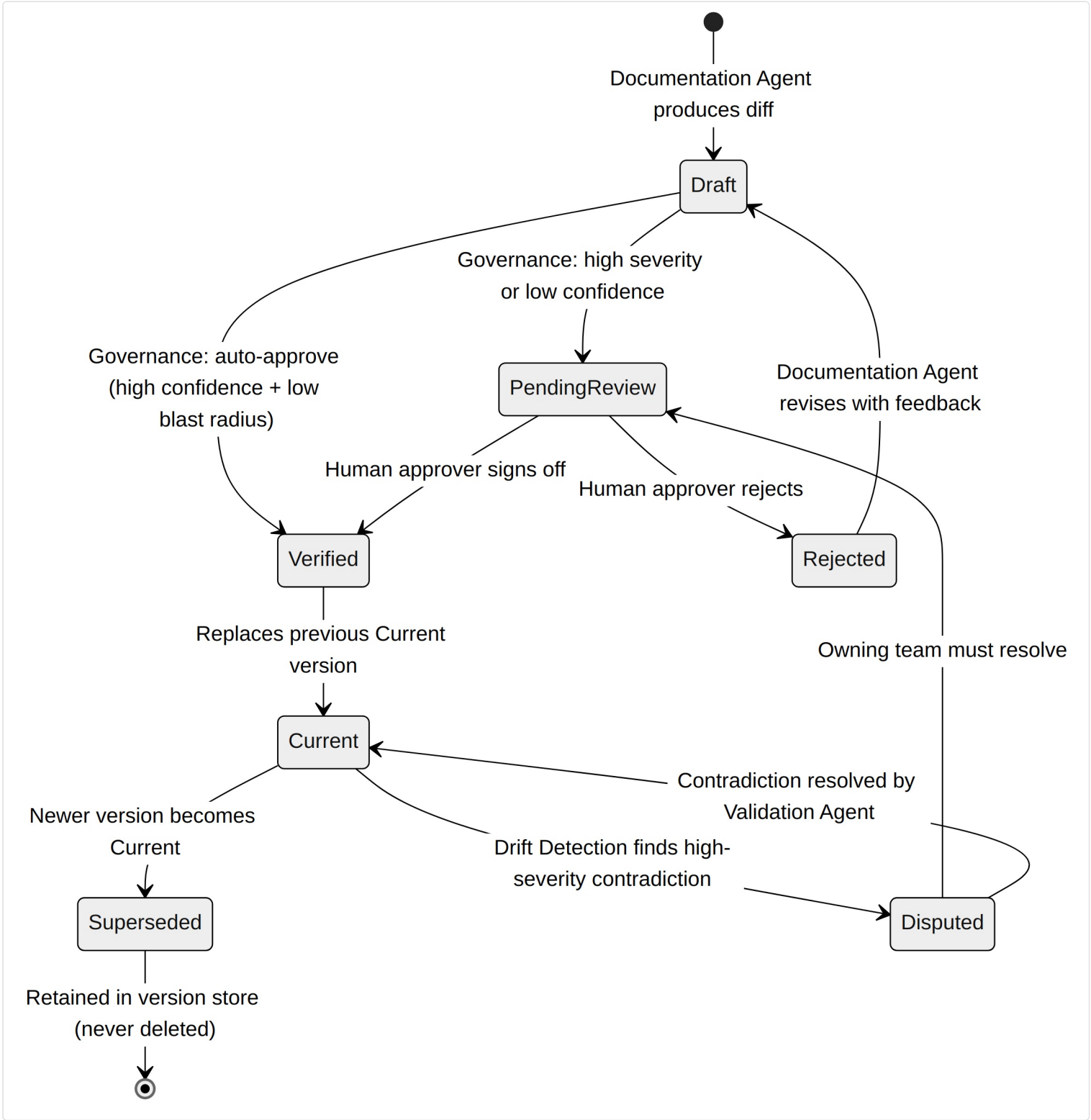


Figure A1 — Version state machine. Every artifact version begins as a Draft and either reaches Current (published) or is Rejected. Published versions can be demoted to Disputed by the Drift Detection Agent. Superseded versions are retained permanently.

State definitions

State	Meaning	Visible to consumers?
Draft	Agent has produced a proposed diff. Not yet approved. Stored in the version store but not published.	No — internal only
PendingReview	Routed to human reviewer because confidence is below threshold or severity is high. The previous Current version remains active but is flagged with an "update pending" banner.	Partially — current version flagged

Verified	Passed governance gate (auto or human). Transitional — becomes Current immediately on approval.	No — instant transition
Current	The live, published version. Only one version per artifact can be Current at a time.	Yes — the default version readers see
Disputed	Drift Detection Agent found a high-severity contradiction in a Current version. The version is still readable but displayed with a visible warning. Requires owning team resolution.	Yes — with warning banner
Superseded	A newer version has become Current. This version is retained in the version store, queryable by date or version number, but not served by default.	Only on explicit historical query
Rejected	Human reviewer rejected the draft, or the draft was discarded due to a conflict with a concurrent human edit (Path C). Feedback is returned to the Documentation Agent for revision.	No — internal only

Critical invariants enforced by the Governance Agent

- Exactly one version per artifact has status `Current` at any moment.
- No version content is mutated in place after it leaves `Draft` — only its status field is updated.
- A version in `Superseded` or `Rejected` state is never automatically deleted.
- A version in `Disputed` state cannot be auto-resolved — a human or the Validation Agent must explicitly act.

A3. How Updates Are Triggered

Updates to any document artifact are driven entirely by events, not by schedule. This is the "living document" property: the artifact responds to the world changing, within minutes rather than on a weekly batch cycle.

Trigger event taxonomy

Event type	Example	Typical artifacts affected
Code commit / PR merge	New database connection added to service A	Architecture Pack (dependency graph, data flows), Developer Pack (setup, build)
CI/CD pipeline change	New deployment target added	Developer Pack (deployment process), Architecture Pack (environment topology)
IaC change	Security group rule modified	Architecture Pack (security boundaries) — auto-routed to PendingReview by default given the sensitivity
Incident opened / closed	INC-4471 closed with postmortem	Operations Pack (common failures, runbooks), Architecture Pack (if root cause reveals undocumented dependency)
ADR / RFC merged	ADR-022: deprecate legacy endpoint	Architecture Pack (affected service entries), Developer Pack (coding standards / API guidelines)
Slack / conversation distilled	SME Interview Agent produces FAQ from thread	Developer Pack (FAQ section), Operations Pack (troubleshooting), Architecture Pack (rationale section)
Human edit on published surface	On-call engineer corrects runbook step during incident	Operations Pack (affected runbook) — always a high-trust, high-priority update
Scheduled drift scan	Weekly full-graph comparison pass	Any artifact where a claim can no longer be corroborated — triggers Disputed status, not a new version
API spec change	New endpoint added to OpenAPI spec	Architecture Pack (API surface in service catalog), AI Agent Pack (available tools if this is an MCP-exposed API)

What does NOT trigger an update

The Discovery Agent filters events before dispatching to the Documentation Agent. Events that do not affect any claim in the knowledge graph — for example, a commit that only changes test data, a comment-only PR, or a Slack message that the NL Extraction Agent scores as low-information — are acknowledged, stored in the raw document store for provenance, and not escalated to the update pipeline. This filter is essential for preventing update noise at high commit velocity.

A4. The Three Update Paths

Every update to a document artifact follows one of three paths, depending on its origin. The sequence diagram below shows all three paths in a single view, including the conflict resolution rule when Path A and Path B collide.

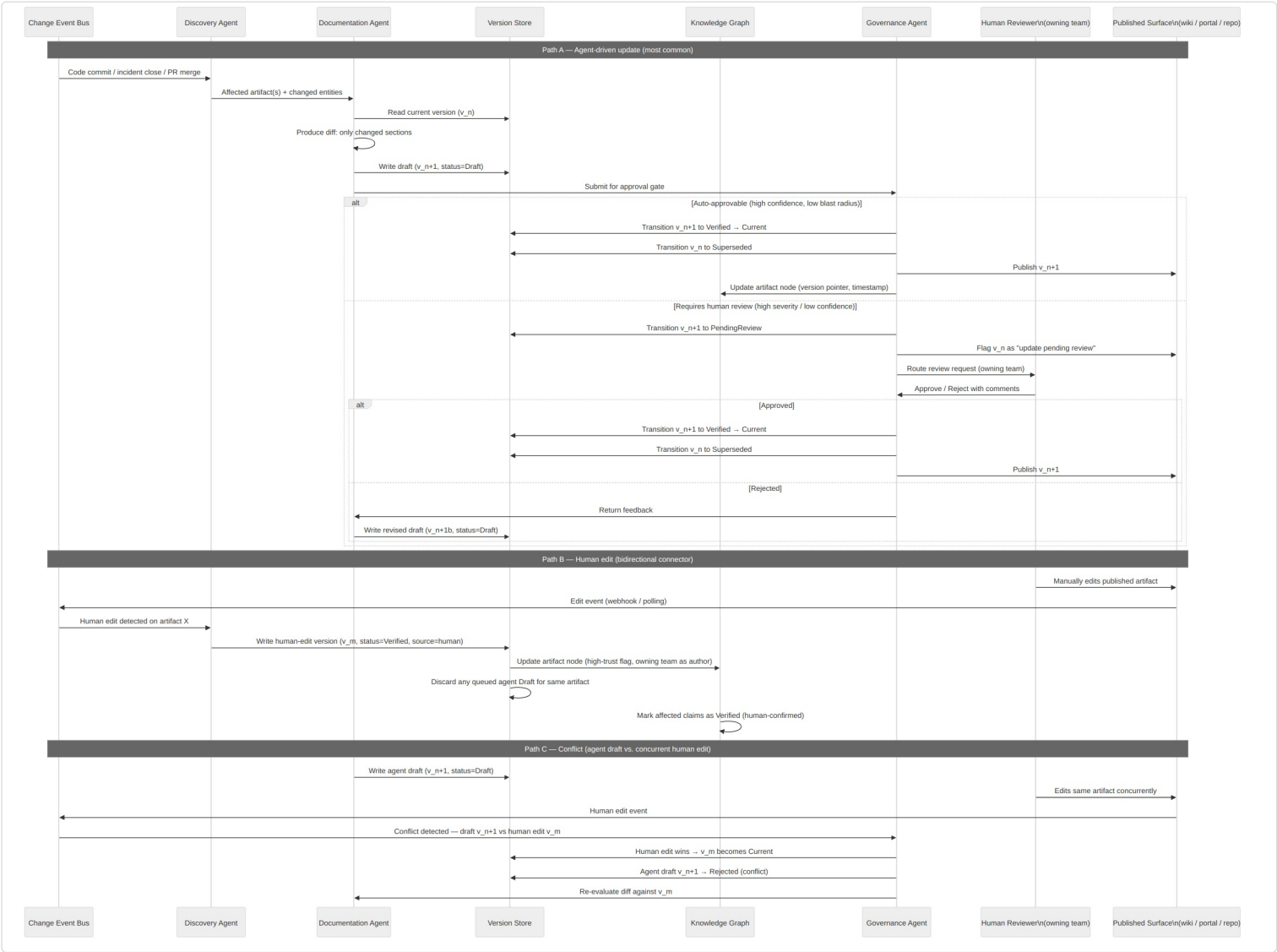


Figure A2 — The three update paths: Path A (agent-driven), Path B (human edit via bidirectional connector), and Path C (conflict between concurrent agent draft and human edit). Path B always takes precedence over Path A in Path C.

Path A — Agent-driven update (most common)

The Documentation Agent reads the current version of the artifact from the version store, identifies which section(s) are affected by the triggering change, and produces a **diff** — not a full rewrite. This diff is written as a new Draft version. The Governance Agent then evaluates it against the auto-approve policy:

- If both confidence score $\geq$ threshold *and* blast radius is low → auto-approve: transition to Verified → Current, transition previous version to Superseded, publish to the relevant surface.
- If either condition fails → transition to PendingReview, flag the current version as "update pending," route a review request to the owning team via their preferred channel (Slack/Teams/ticket).

Path B — Human edit (bidirectional connector)

A human edits a published document directly in the wiki, portal, or runbook repository. The bidirectional connector for that surface detects the edit via webhook or polling and fires a change event back to the system. The Discovery Agent recognizes this as a human edit on artifact X and writes it immediately as a new version with:

- `source: human`
- `confidence: 1.0` (human-authoritative, specifically the owning team)
- `human-reviewed: true`, with the editor's identity from the surface's auth context

Critically, any queued agent Draft for the same artifact is **discarded immediately** — not routed through governance, not

merged, simply voided. The knowledge graph is also updated: all claims derived from the affected section are re-evaluated against the new human-provided text and marked Verified (human-confirmed).

Path C — Conflict (agent draft vs. concurrent human edit)

If an agent Draft exists in the version store at the moment a human edit event arrives for the same artifact, the Governance Agent detects the collision and applies the following deterministic resolution:

1. The human edit (v_m) is written and becomes Current immediately.
2. The agent draft (v_n+1) is transitioned to Rejected with reason `conflict:human-edit`.
3. The Documentation Agent is asked to re-evaluate its proposed changes against v_m as the new baseline — some of the agent's proposed changes may still be valid and will be re-submitted as a new Draft against v_m.

Why human always wins in a conflict. The primary risk in the AKES design (inherited from the Netflix principle cited in the main brief) is wrong-but-confident output reaching consumers. A human edit from the owning team represents the highest possible trust signal. Merging agent and human edits would introduce the risk of an agent's drift-unaware change silently overriding a human correction — precisely the failure mode the entire system exists to prevent.

A5. Version Store — Schema & Provenance

The version store is an append-only log of artifact versions. It is separate from the knowledge graph (which stores entities and relationships) and from the raw document store (which stores original source artifacts). Its purpose is to be the authoritative record of what was published, when, why, and by whom.

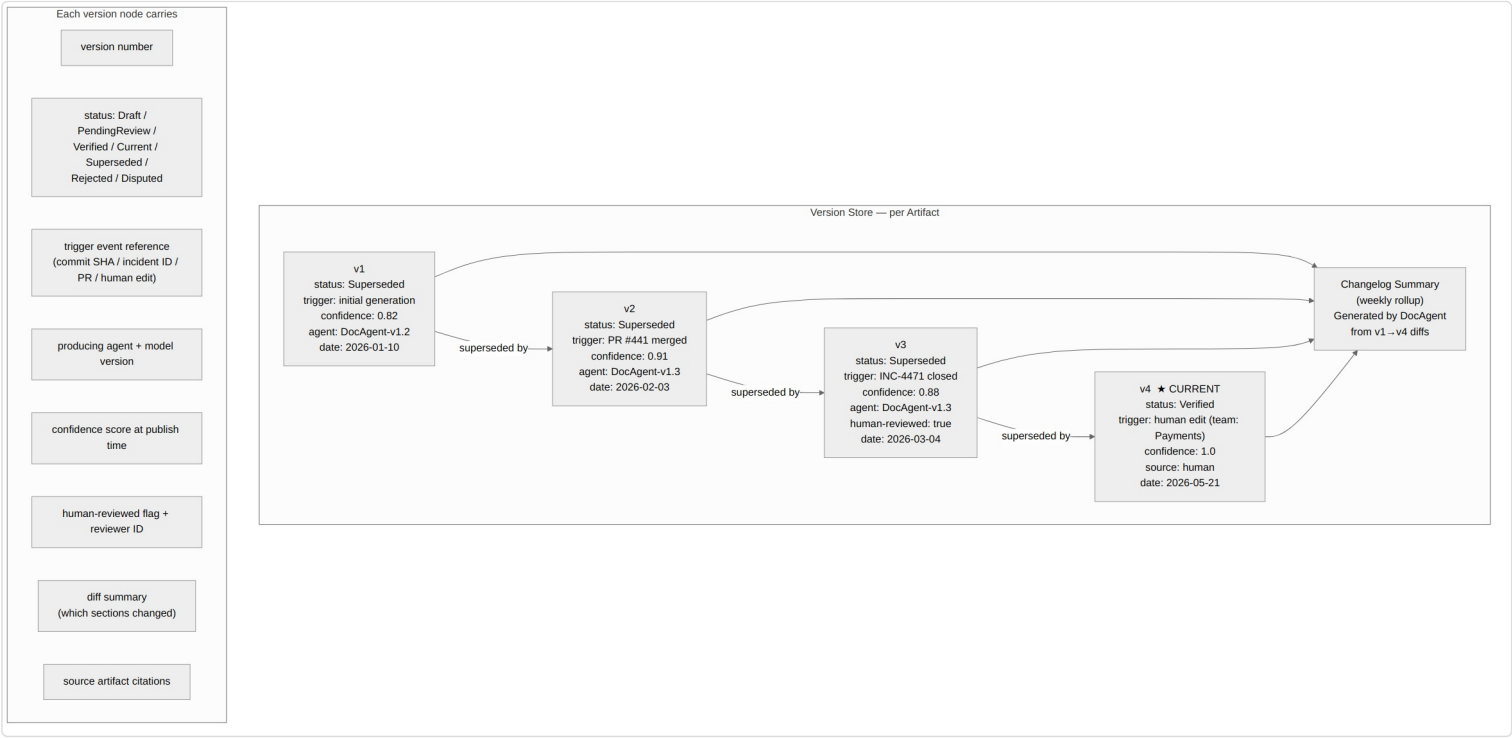


Figure A3 — Version store structure for a single artifact, showing four versions across five months and the weekly changelog rollout generated from their diffs. Left panel shows the per-version metadata schema.

Per-version metadata schema

Field	Type	Purpose
version_id	UUID	Globally unique; not a sequential integer, to avoid distributed coordination on ID generation
artifact_id	UUID	Foreign key to the artifact entity in the knowledge graph
version_number	Integer (monotonic per artifact)	Human-readable sequence for display ("v4 of payments-runbook")
status	Enum	Current state in the lifecycle (Draft / PendingReview / Verified / Current / Superseded / Rejected / Disputed)
content_hash	SHA-256	Hash of the full document content at this version — enables tamper detection and deduplication
diff_from_prev	Structured diff (section-level)	What changed from the previous version, stored as a structured diff rather than raw text — enables section-level queries ("what changed in the runbook's recovery procedure section?")
trigger_event_ref	String	Reference to the originating change event (commit SHA, incident ID, PR number, "human:wiki-edit:user@org")
producing_agent	String	Agent role + model version that produced this version (e.g., DocAgent/claude-sonnet-4-6). human for Path B edits.
confidence_score	Float [0.0–1.0]	Confidence at time of production. 1.0 for human-origin versions.
source_citations	Array of refs	Source artifact references used to produce/validate this version (commit, Slack thread, ADR, postmortem)
human_reviewed	Boolean	True if a human reviewer explicitly approved (PendingReview → Verified path) or authored (Path B)
reviewer_id	String (optional)	Identity of the human reviewer or editor, from auth context
governance_policy_id	String	Which auto-approve policy rule was applied (if auto-approved), for audit calibration
created_at	Timestamp (UTC)	When this version was written to the store

published_at	Timestamp (UTC, nullable)	When this version became Current — null for Rejected/Draft versions
superseded_at	Timestamp (UTC, nullable)	When a newer version replaced this one as Current

Why content_hash matters

The content hash serves two purposes. First, it enables the version store to detect if two distinct change events produce semantically identical content — a common case when two unrelated commits both touch the same service's entry in the catalog — allowing the system to skip publishing an identical version. Second, it provides a tamper-evident record: if a version's stored content no longer matches its hash, the system knows it was modified outside the normal write path, which should be treated as a security event.

A6. Changelog Rollup & Consumer-Facing Views

At high change velocity, a runbook or architecture document may accumulate dozens of micro-versions per week — each one tracking a precise, small change. This granularity is valuable for audit and incident investigation, but it is too noisy for day-to-day consumers who simply want to know "what changed this sprint?"

Changelog rollup

The Documentation Agent runs a weekly changelog rollup job per artifact. It reads all version diffs since the last rollup and produces a human-readable summary: "This week: added dependency on Queue C (v12, PR #882), updated recovery procedure for DB failover (v13, INC-4502)." This changelog is itself a versioned artifact stored in the version store, with its own provenance (the set of version IDs it summarizes).

The rollup is also available programmatically — agents consuming the knowledge graph via MCP can query "what changed in artifact X between date A and date B?" and receive a structured diff summary rather than iterating through individual versions.

Consumer-facing version indicators

Published surfaces (developer portal, wiki, on-call dashboards) should surface the following metadata alongside every document, drawing directly from the version store:

Indicator	What it shows	Why it matters
Version badge	"v14 — last updated 3 days ago"	Immediately signals whether a document is recent or potentially stale relative to the reader's context
Trust level badge	Verified / Disputed / Pending Review	The core governance signal — readers know whether to act on the document with full confidence or treat it as provisional
Trigger summary	"Updated after: PR #882 (payments-api)"	Allows readers to immediately understand why the document changed without opening the version history
Change history link	Link to full version list with diffs	Supports incident investigation — "what did this runbook say before the outage?"
Pending update banner	"An update is pending human review — this document may not reflect recent changes."	Prevents consumers from acting on a document that the system already knows is outdated but cannot yet auto-approve

A7. Partial Updates — Section-Level Granularity

One of the three core design properties stated in A1 is that updates are diff-based, not full rewrites. This requires the Documentation Agent to understand artifacts at the section level, not just as flat text blobs. This section details how that works and why it matters.

Section model

Each knowledge pack artifact is defined by a stable section schema registered in the knowledge graph. For example, the Architecture Knowledge Pack has sections: *System Overview*, *Service Catalog*, *Dependency Graph*, *Data Flows*, *Security Boundaries*. Each section maps to a set of knowledge graph entity types and relationship types that it renders. When a change event affects only the *Dependency Graph* section (e.g., a new service-to-service call detected), only that section is re-rendered and diffed against the current version — the other sections are copied forward unchanged.

This has three important consequences:

- **Human edits in unaffected sections are preserved.** If an on-call engineer manually annotated the Security Boundaries section of an architecture doc with a note about a known exception, and the next agent update only touches the Dependency Graph section, that annotation survives untouched. The Documentation Agent never re-renders sections it was not asked to update.
- **Blast radius of updates is bounded.** The Governance Agent evaluates severity and confidence per-section, not per-document. A high-confidence, low-blast-radius change to the Service Catalog section of the Architecture Pack can be auto-approved even if the same document has a Pending Review update in progress for the Security Boundaries section.
- **Version diffs are semantically meaningful.** The diff stored in the version store is section-structured — "Security Boundaries: unchanged, Dependency Graph: added edge (payments-api → fraud-queue), Data Flows: unchanged" — rather than a line-level text diff. This is what makes the changelog rollup (Section A6) legible without further LLM processing.

Implementation note. The section schema for each pack is itself a versioned artifact in the system. If the Architecture Pack gains a new "Environment Topology" section, that schema change is versioned and the previous section schema is retained — so historical versions of the pack can still be rendered correctly using the schema that was active when they were published.

A8. Version Retention & Compaction Policy

The version store is append-only and versions are never deleted — but at scale, storing the full content of every micro-version of hundreds of artifacts indefinitely would become impractical. The retention model addresses this through a tiered storage policy that preserves permanent access to key versions while compacting intermediate ones.

Retention tiers

Tier	Versions retained	What is kept
Full fidelity (permanent)	All <code>Current</code> versions All <code>Disputed</code> resolutions Any version referenced by an incident or postmortem Any human-reviewed version First version of any artifact (genesis)	Full content + all metadata + diff from previous
Metadata-only (permanent)	All <code>Rejected</code> versions All <code>Superseded</code> versions older than 90 days not referenced by an incident	All metadata fields (see Section A5 schema), content hash, diff summary — but NOT full content body. Content can be reconstructed from the genesis version + the ordered chain of diffs.
Purged	Draft versions older than 30 days that never left Draft status	Only <code>version_id</code> , <code>artifact_id</code> , <code>created_at</code> , and <code>status: Draft (purged)</code> retained as a tombstone

Incident reference lock

When an incident or postmortem document is created, the system queries the version store for all artifacts that were in `Current` status at the incident start time and records those version IDs in the incident entity in the knowledge graph. This "snapshot at incident time" is locked — those versions are permanently pinned to Full Fidelity retention, regardless of age, because they are potentially needed for future postmortem review, regulatory audit, or comparative analysis ("did the runbook change before or after this class of incident started recurring?").

A9. Design Decisions (ADR-style)

ADR-V1: Immutable versions vs. in-place mutation with change tracking

Decision: Every version is immutable once it leaves Draft. Changes produce a new version.

Alternatives considered: A single mutable document with tracked changes (similar to Word track-changes or Google Docs history). This is the model most wiki tools use.

Trade-off accepted: Immutable versions require more storage and a more complex data model than tracked-changes mutation. In exchange, any version is independently addressable by ID, the content hash is a stable integrity guarantee, concurrent edits cannot produce merge conflicts within a single document blob, and incident snapshots are trivial to implement (just reference version IDs).

ADR-V2: Section-level diffs vs. full document regeneration

Decision: The Documentation Agent re-renders only the sections affected by a triggering change event.

Alternatives considered: Full document regeneration on every update — simpler to implement and avoids the need for a stable section schema.

Trade-off accepted: Requires defining and versioning a section schema per pack (Section A7 implementation note), which is non-trivial. In exchange, human edits in unaffected sections are preserved across agent updates, blast radius of auto-approval is scoped to sections rather than whole documents, and diffs in the version store are semantically meaningful rather than line-level noise.

ADR-V3: Human edits always win in conflict vs. merge strategy

Decision: In a conflict between a concurrent agent draft and a human edit, the human edit always wins and the agent draft is discarded.

Alternatives considered: An LLM-mediated merge that attempts to incorporate both the agent's proposed changes and the human's manual edits into a single new version, routed for human approval.

Trade-off accepted: The discard-and-re-evaluate approach means some valid agent changes may be delayed (they are re-submitted against the human-edited baseline). The merge approach could produce a higher rate of incorporating valid agent changes, but introduces the risk that the merge result is subtly wrong in ways neither the human nor the agent would independently produce — exactly the wrong-but-confident failure mode the system is designed to prevent. Discard-and-re-evaluate is the safe default; if agent change loss rates prove problematic in practice, the merge approach can be introduced cautiously for specific section types.

ADR-V4: Content hash for deduplication and tamper detection

Decision: Every version stores a SHA-256 hash of its content at creation time, used for deduplication and integrity checking.

Alternatives considered: Relying on version IDs and timestamps alone; using a weaker hash (MD5/SHA-1) for performance.

Trade-off accepted: SHA-256 is computationally negligible for document-sized content. The deduplication benefit (skipping identical re-publications at high commit velocity) is significant. The tamper-detection benefit is low-probability but high-value — a document that was silently modified outside the normal write path during a security incident is exactly the kind of thing a post-incident audit needs to be able to detect.

A10. Integration with the Governance & Trust Model

The versioning model in this addendum is the operational implementation of the trust model described in Section 10 of the main AKES brief. The two interlock at three specific points:

1. Trust level is a property of the version, not the artifact

An artifact's current trust level (Verified, Disputed, Pending Review, etc.) is derived from its Current version's status in the version store. This means trust level is automatically historical — if you query a previous version, you get its trust level at the time it was Current, not today's trust level. This is critical for incident retrospectives.

2. Auto-approve threshold calibration uses version history

The Governance Agent's auto-approval policy is not static. It is calibrated quarterly using the version history: "For agent-produced versions of type Architecture Pack / Dependency Graph section, what fraction of auto-approved versions were subsequently edited by a human reviewer within 30 days?" A high rate of post-publish human edits is a signal that the auto-approve threshold for that combination of pack type + section is too permissive and should be tightened. This closes the governance feedback loop using data the version store naturally accumulates.

3. Disputed status is the bridge between drift detection and versioning

When the Drift Detection Agent finds a high-severity contradiction in a Current version, it does not create a new version — it transitions the Current version's status to Disputed. This is a deliberate design choice: the document has not changed, but its trust status has. The resolution of a Dispute — either the Validation Agent confirms the current content is still correct, or the Documentation Agent produces a corrected new Draft — is what eventually produces a new version. This keeps the version history clean (a Dispute is a status event, not a content event) while ensuring that trust degradation is immediately visible to consumers.

Closing principle. The versioning model described in this addendum operationalizes a single idea: *every claim in a trusted knowledge artifact should be traceable to a source, a time, an agent or human, and a confidence level* — and that trace should be queryable by any consumer, human or agent, at any point in time. This is what makes "trusted" a system property rather than a label.